Classification Assignment

Problem Statement or Requirement:

A requirement from the Hospital, Management asked us to create a predictive model which will predict the Chronic Kidney Disease (CKD) based on the several parameters. The Client has provided the dataset of the same.

1.) **Identify your problem statement**

Solution:

To predict the Chronic Kidney Disease (CKD)

2.) **Tell basic info about the dataset (Total number of rows, columns)**

Solution:

The total number of rows and columns in dataset are 339 rows x 25 columns

3.) **Mention the pre-processing method if you're doing any (like converting string to number – nominal data)**

Solution:

The pre-processing method is **pd.get_dummies** () for converting Sex and smoker in insurance_pre dataset

4.) Develop a good model. You can use any machine learning algorithm; you can create many models. Finally, you have to come up with final model

Solution:

Yes ,I develop many models with confusion matrix and push into git hub as classification assignment .

5.) All the research values should be documented.

(You can make tabulation or screenshot of the results.)

Solution:

1) Logistic Regression

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_penalty	param_solver	params	split0_
0	0.010030	0.001246	0.004601	0.001124	l2	newton-cg	{'penalty': 'l2', 'solver': 'newton-cg'}	
1	0.007626	0.001470	0.003925	0.000121	l2	lbfgs	{'penalty': 'l2', 'solver': 'lbfgs'}	
2	0.003961	0.000597	0.004112	0.001097	l2	liblinear	{'penalty': 'l2', 'solver': 'liblinear'}	
3	0.019996	0.005607	0.003231	0.000305	l2	saga	{'penalty': 'l2', 'solver': 'saga'}	

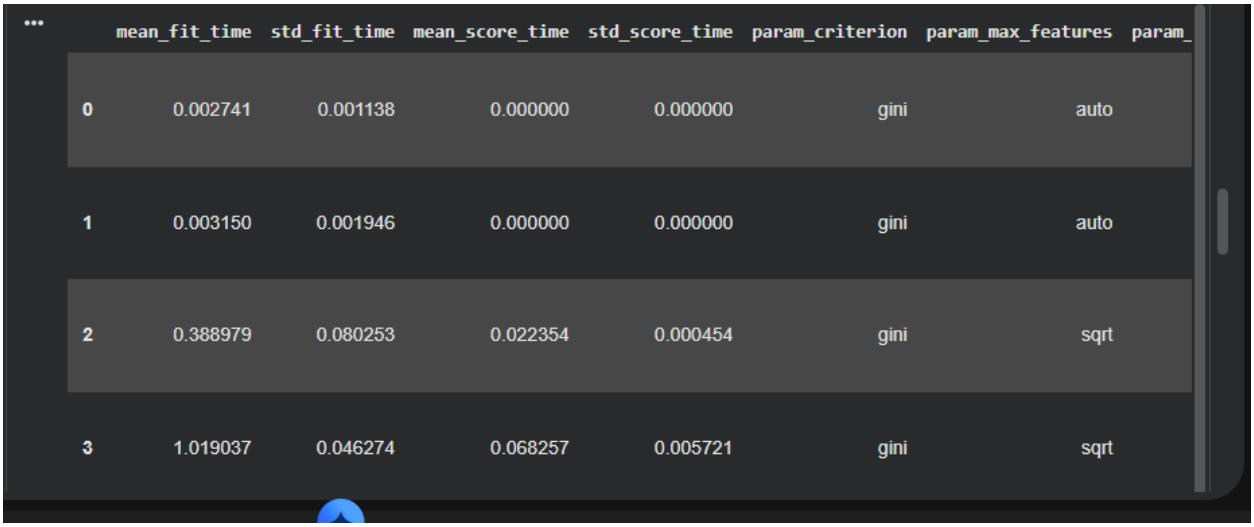
	precision	recall	f1-score	support
0	0.98	1.00	0.99	51
1	1.00	0.99	0.99	82
accuracy			0.99	133
macro avg	0.99	0.99	0.99	133
weighted avg	0.99	0.99	0.99	133

2. K Nearest Neighbor

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_metric	param_n_neighbors	param_weig
0	0.006000	0.002260	0.012209	0.002314	euclidean	3	unif
1	0.007988	0.003836	0.009954	0.004173	euclidean	3	dista
2	0.010250	0.002467	0.015973	0.004999	euclidean	5	unif
3	0.005021	0.002516	0.011179	0.009857	euclidean	5	dista

	precision	recall	f1-score	support
0	0.94	1.00	0.97	51
1	1.00	0.96	0.98	82
accuracy			0.98	133
macro avg	0.97	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

3. Naïve Bayes



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_
0	0.002741	0.001138	0.000000	0.000000	gini	auto	
1	0.003150	0.001946	0.000000	0.000000	gini	auto	
2	0.388979	0.080253	0.022354	0.000454	gini	sqrt	
3	1.019037	0.046274	0.068257	0.005721	gini	sqrt	

	precision	recall	f1-score	support
0	0.98	0.98	0.98	51
1	0.99	0.99	0.99	82
accuracy			0.98	133
macro avg	0.98	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

4. Random Forest

```
table=pd.DataFrame.from_dict(grid.cv_results_)
table
```

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	param_
0	0.002741	0.001138	0.000000	0.000000	gini	auto	
1	0.003150	0.001946	0.000000	0.000000	gini	auto	
2	0.388979	0.080253	0.022354	0.000454	gini	sqrt	
3	1.019037	0.046274	0.068257	0.005721	gini	sqrt	

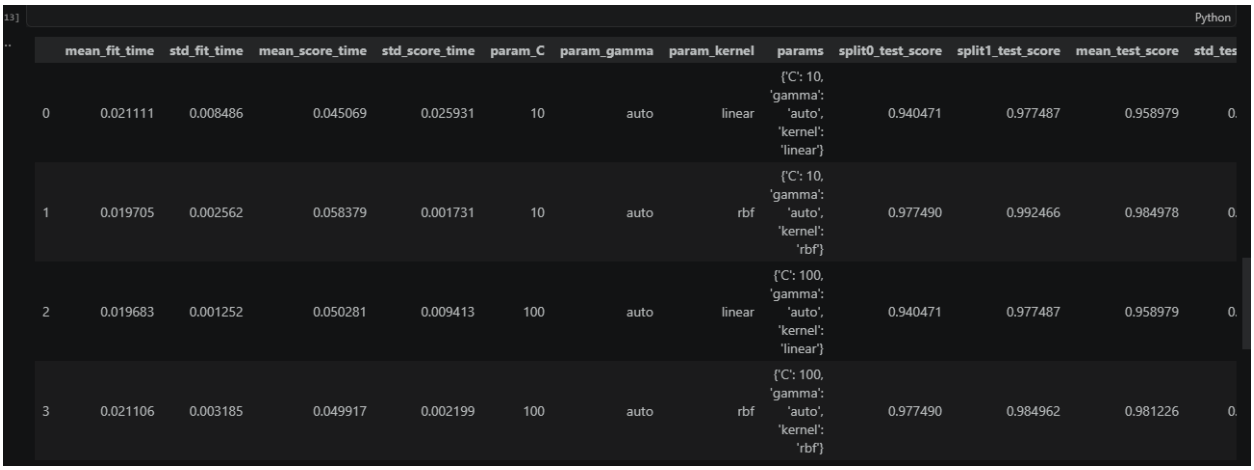
	precision	recall	f1-score	support
0	0.98	0.98	0.98	51
1	0.99	0.99	0.99	82
accuracy			0.98	133
macro avg	0.98	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

5.Decision Tree Classification

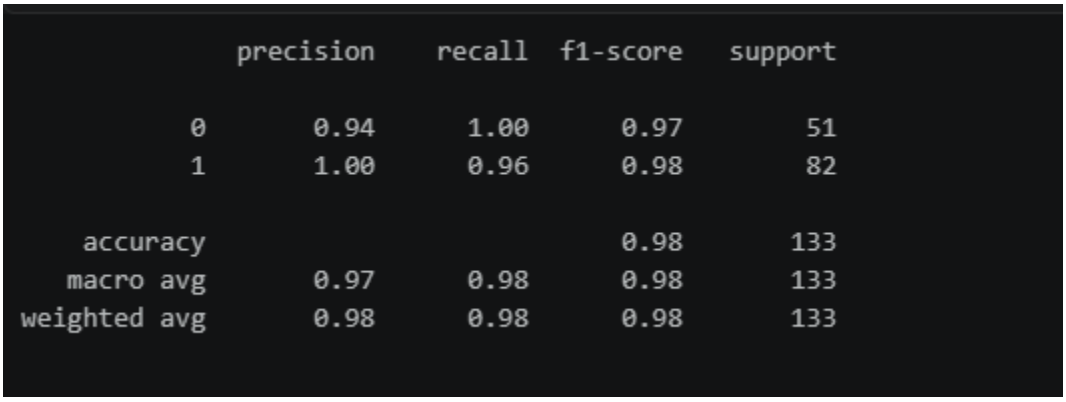
	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_criterion	param_max_features	params	split0_test_score	split1_test_score	split2_test_score	split
0	0.002431	0.001320	0.000000	0.000000	gini	auto	{'criterion': 'gini', 'max_features': 'auto'}	NaN	NaN	NaN	
1	0.009427	0.004222	0.023380	0.009511	gini	sqrt	{'criterion': 'gini', 'max_features': 'sqrt'}	0.962963	0.903610	0.962573	
2	0.005846	0.002462	0.013988	0.005119	gini	log2	{'criterion': 'gini', 'max_features': 'log2'}	0.890467	0.943699	0.868519	
3	0.001940	0.001119	0.000000	0.000000	entropy	auto	{'criterion': 'entropy', 'max_features': 'auto'}	NaN	NaN	NaN	
4	0.005756	0.003225	0.012684	0.007656	entropy	sqrt	{'criterion': 'entropy', 'max_features': 'sqrt'}	1.000000	0.905069	0.925146	
5	0.004749	0.002701	0.012626	0.008434	entropy	log2	{'criterion': 'entropy', 'max_features': 'log2'}	0.981569	0.961755	0.944023	

	precision	recall	f1-score	support
0	0.92	0.96	0.94	51
1	0.97	0.95	0.96	82
accuracy			0.95	133
macro avg	0.95	0.96	0.95	133
weighted avg	0.96	0.95	0.96	133

6. Support Vector Machine



	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_C	param_gamma	param_kernel	params	split0_test_score	split1_test_score	mean_test_score	std_tes
0	0.021111	0.008486	0.045069	0.025931	10	auto	linear	{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.940471	0.977487	0.958979	0.
1	0.019705	0.002562	0.058379	0.001731	10	auto	rbf	{'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}	0.977490	0.992466	0.984978	0.
2	0.019683	0.001252	0.050281	0.009413	100	auto	linear	{'C': 100, 'gamma': 'auto', 'kernel': 'linear'}	0.940471	0.977487	0.958979	0.
3	0.021106	0.003185	0.049917	0.002199	100	auto	rbf	{'C': 100, 'gamma': 'auto', 'kernel': 'rbf'}	0.977490	0.984962	0.981226	0.



	precision	recall	f1-score	support
0	0.94	1.00	0.97	51
1	1.00	0.96	0.98	82
accuracy			0.98	133
macro avg	0.97	0.98	0.98	133
weighted avg	0.98	0.98	0.98	133

6.) Mention your final model, justify why u have chosen the same

Solution:

The final model is using **Logistic Rgression** because it predict the accuracy and f1_score is **0.99**